





*Serviço Nacional
de Aprendizagem
Industrial*

Introdução à Orientação a Objetos

Programação de Aplicativos

Introdução à Orientação a Objetos

- **Instrutor:** Lucas Ribeiro Arêas
- **Unidade Curricular:** Programação de Aplicativos
- **Curso:** Técnico em Desenvolvimento de Sistemas

SENAI Vitória - DR/ES

Introdução

Introdução

Todos os sistemas de computação, dos mais complexos aos mais simples programas de computadores, são desenvolvidos para auxiliar na solução de problemas do mundo real.

Introdução

Mas, de fato, o mundo real é extremamente complexo. Nesse contexto, a orientação a objetos tenta gerenciar a complexidade inerente aos problemas do mundo real abstraindo o conhecimento relevante e encapsulando-o dentro de objetos.

Introdução

Na programação orientada a objetos um programa de computador é conceituado como um conjunto de objetos que trabalham juntos para realizar uma tarefa. Cada objeto é uma parte do programa, interagindo com as outras partes de maneira específica e totalmente controlada.

Introdução

Na visão da orientação a objetos, o mundo é composto por diversos objetos que possuem um conjunto de características e um comportamento bem definido.

Introdução

Assim, quando programamos seguindo o paradigma de orientação a objetos, definimos abstrações dos objetos reais existentes. Todo objeto possui as seguintes características:

Introdução

- **Estado:** conjunto de propriedades de um objeto (valores dos atributos).
- **Comportamento:** conjunto de ações possíveis sobre o objeto (métodos da classe).
- **Unicidade:** todo objeto é único (possui um endereço de memória).

Introdução

Quando você escreve um programa em uma linguagem orientada a objetos, não define objetos individuais. Em vez disso define as classes utilizadas para criar esses objetos.

Introdução

Assim, podemos entender uma **classe** como um modelo ou como uma especificação para um conjunto de objetos, ou seja, a descrição genérica dos objetos individuais pertencentes a um dado conjunto. A partir de uma **classe** é possível criar quantos objetos forem desejados.

Introdução

Uma **classe** define as características e o comportamento de um conjunto de objetos. Assim, a criação de uma **classe** implica definir um tipo de objeto em termos de seus atributos (variáveis que conterão os dados) e seus métodos (funções que manipulam tais dados).

Introdução

Já um **objeto** é uma **instância** de uma classe. Ele é capaz de armazenar estados por meio de seus atributos e reagir a mensagens enviadas a ele, assim como se relacionar e enviar mensagens a outros objetos.

Introdução

Joana, José e Maria, por exemplo, podem ser considerados objetos de uma classe **Pessoa** que tem como atributos *nome*, *CPF*, *data de nascimento*, entre outros, e métodos como *calcularIdade*.

Introdução

Quando desenvolvemos um sistema orientado a objetos, definimos um conjunto de classes. Para organizar essas classes surge o conceito de **pacote**.

Introdução

Pacote é um envoltório de classes, ou seja, guarda classes e outros pacotes logicamente semelhantes ao pacote que os contém.

Introdução

Podemos visualizar os pacotes como diretórios ou pastas, nos quais podemos guardar arquivos (classes) e outros diretórios (pacotes) que tenham conteúdo relacionado com o pacote que os contém.

1 - Classes no Java

Classes no Java

Sabemos que para desenvolver programa orientado a objetos precisamos criar classes. Uma classe é como um tipo de objeto que pode ser definido pelo programador para descrever uma entidade real ou abstrata. Mas, como definir classes em Java?

Classes no Java

Veja a seguir como é a sintaxe de uma classe no Java:

Classes no Java

```
<qualificador> class <nome_da_classe> {  
    <declaração dos atributos da classe>  
    <declaração dos métodos da classe>  
}
```

Classes no Java

<qualificador>: O qualificador de acesso determinará a visibilidade da classe. Pode ser ***public*** (classe pública) ou ***private*** (classe privada).

Classes no Java

Classes privadas só poderão ser visualizadas dentro de seu próprio pacote enquanto as públicas serão acessíveis por qualquer classe de qualquer pacote. Se o qualificador for omitido, a classe será privada por padrão.

Classes no Java

`<nome_da_classe>`: nome que identifica a classe. Há um padrão entre os programadores de sempre iniciarem os nomes de classes com letras maiúsculas. Mas, apesar de ser uma boa prática, seguir esse padrão não é uma obrigação.

Classes no Java

Vejam os a seguir a criação de três atributos da classe exemplo chamada **Conta** em Java.

Classes no Java

```
1 package exemplos;  
2  
3 public class Conta {  
4     int numero;  
5     String nome_titular;  
6     double saldo;  
7 }
```

Classes no Java

No exemplo a seguir, veremos a criação de duas classes, distintas apenas com relação ao seu qualificador.

Classes no Java

```
package pacote1;  
  
class ClassePrivada {  
    int atributo1;  
}
```

```
package pacote1;  
  
public class ClassePublica {  
    int atributo1;  
}
```

Classes no Java

A diferença será notada na hora em que estas classes forem utilizadas. Como foi dito anteriormente, classes privadas só podem ser referenciadas (vistas) por outras classes do mesmo pacote.

Classes no Java

Já as classes públicas podem ser referenciadas por outras classes de pacotes diferentes sem causar erro no código, conforme podemos observar a seguir com a classe privada.

Classes no Java

```
package pacote2;  
  
import pacote1.ClassePublica;  
import pacote1.ClassePrivada;  
  
public class ClasseOutroPacote {  
    ClassePublica obj1;  
    ClassePrivada obj2;  
}
```


Classes no Java

Um detalhe importante de se perceber é que precisamos sempre importar uma classe originária de outro pacote toda vez que esta for utilizada.

2 - Declaração de Atributos e Métodos

Declaração de Atributos e Métodos

Como vimos anteriormente, sempre que quando criamos uma classe em Java precisamos declarar os atributos e métodos da classe.

Declaração de Atributos e Métodos

Vejamos a seguir a sintaxe para declarar um atributo de uma classe:

Declaração de Atributos e Métodos

<qualificador> <tipo_do_atributo> <nome_do_atributo>;

Declaração de Atributos e Métodos

<qualificador>: o qualificador de acesso determinará a visibilidade do atributo. É opcional e, se não for informado, por padrão o atributo será protegido (*protected*).

Declaração de Atributos e Métodos

<tipo_do_atributo>: é um tipo primitivo ou classe que define o atributo.

Declaração de Atributos e Métodos

`<nome_do_atributo>`: nome que identifica o atributo. Há um padrão entre os programadores de sempre iniciarem os nomes de atributos com letras minúsculas. Mas, apesar de ser uma boa prática, seguir esse padrão não é uma obrigação. Caso queiramos definir vários atributos de mesmo tipo podemos colocar os vários nomes separados por vírgula.

Declaração de Atributos e Métodos

Alguns exemplos de declaração de atributos:

- *private* **double** saldo;
- **String** nome, endereço;

Declaração de Atributos e Métodos

Para declararmos um método, utilizamos a seguinte sintaxe:

Declaração de Atributos e Métodos

```
<qualificador> <tipo_do_retorno>  
<nome_do_método>(<lista_de_argumentos>){  
  
    <corpo_do_método>  
  
}
```

Declaração de Atributos e Métodos

<qualificador>: O qualificador de acesso determinará a visibilidade do método. É opcional e, se não for informado, por padrão o método será protegido (*protected*).

Declaração de Atributos e Métodos

<tipo_do_retorno>: é um tipo primitivo ou classe que define o retorno a ser dado pelo método.

Declaração de Atributos e Métodos

<nome_do_método>: nome que identifica o método.

<corpo_do_método>: código que define o que o método faz.

Declaração de Atributos e Métodos

Exemplo de declaração de método:

- *public* **double** getSaldo(){
 return saldo;
}

Declaração de Atributos e Métodos

Vejam os nosso exemplo da classe **Conta** com os seus métodos e atributos:

Declaração de Atributos e Métodos

```
package exemplos;

public class Conta {

    int numero;
    String nome_titular;
    double saldo;

    void depositar(double valor) {
        this.saldo = this.saldo + valor;
    }
}
```

Declaração de Atributos e Métodos

O método ***depositar*** recebe como argumento o valor a ser depositado na conta e soma esse valor ao saldo da conta. Note que no lugar do tipo de retorno do método foi utilizada a palavra *void*, que indica que nenhum valor será retornado pelo método.

Declaração de Atributos e Métodos

Outro detalhe interessante no exemplo anterior é o uso da palavra *this*. A palavra *this* é utilizada para acessar atributos do objeto corrente.

Declaração de Atributos e Métodos

A palavra *this* é obrigatória apenas quando temos um argumento ou variável local de mesmo nome que um atributo. Entretanto, é recomendável sua utilização, mesmo quando não obrigatório, por uma questão de padronização.

Declaração de Atributos e Métodos

Agora, vamos adicionar um outro método importante para a classe **Conta**, o método *sacar*.

Declaração de Atributos e Métodos

```
boolean sacar(double valor) {  
    if (this.saldo >= valor) {  
        this.saldo -= valor;  
        return true;  
    }  
    else  
        return false;  
}
```

Declaração de Atributos e Métodos

Vamos analisar o método *sacar* e entender o que ele faz?!

Declaração de Atributos e Métodos

O método ***sacar*** recebe como argumento o valor a ser sacado. Então, o método verifica se a conta tem saldo suficiente para permitir o saque.

Declaração de Atributos e Métodos

Se o saldo for maior ou igual ao valor do saque, o valor do saque é descontado do saldo e o método retorna o valor *true* informando que o saque ocorreu com sucesso.

Declaração de Atributos e Métodos

Caso contrário, o método apenas retorna o valor *false*, indicando que o saque não pôde ser efetuado. Por isso, o tipo de retorno do método é *boolean*.

3 - Utilizando Objetos

Utilizando Objetos

Para utilizarmos um objeto em Java,
precisamos executar dois passos
importantes:

Utilizando Objetos

- **Declarar uma variável que referenciará o objeto:** assim como fazemos com tipos primitivos, é necessário declarar o objeto.

Utilizando Objetos

- **Instanciar o objeto:** alocar o objeto em memória. Para isso utilizamos o comando *new* e um construtor.

Utilizando Objetos

Vejamos esses dois passos utilizados em um outro código que utiliza a classe **Conta** mostrada anteriormente:

Utilizando Objetos

```
1 package exemplos;
2
3 public class Programa {
4
5     public static void main(String[] args) {
6         Conta c; //Declarando a variável que conterà a referência ao objeto
7         c = new Conta(); //Instanciando um objeto em memória
8         c.nome_titular = "Zé";
9         c.depositar(100);
10        System.out.println("Titular: " + c.nome_titular);
11        System.out.println("Saldo Atual: " + c.saldo);
12    }
13 }
```


Utilizando Objetos

A classe **Conta** tem por objetivo mapear os dados e comportamentos de uma conta bancária. Assim, ela não terá um método *main*. Esse método estará na classe principal do nosso programa.

Utilizando Objetos

A classe **Programa** mostrada que vimos no exemplo anterior é que representa esse nosso programa principal e é nela que utilizaremos a classe **Conta**.

Utilizando Objetos

É possível observar que na linha 6 do código da classe **Programa** é declarada uma variável de nome c que referenciará um objeto da classe **Conta** enquanto na linha 7 o objeto é instanciado em memória.

Utilizando Objetos

Para instanciar um objeto em memória utilizamos o operador *new* (o mesmo que utilizamos para alocar vetores) acompanhado pelo construtor ***Conta()***.

Utilizando Objetos

Na linha 8 é atribuído o valor “Zé” ao atributo *nome_titular* da conta instanciada e, na linha 9, há uma chamada ao método *depositar* passando como argumento o valor 100. Preste atenção ao ponto que foi utilizado nas linhas 8 (*c.nome_titular*) e 9 (*c.depositar(100)*).

Utilizando Objetos

É assim que acessamos métodos e atributos de um objeto em Java. Nas linhas 10 e 11 do código da classe **Programa** há dois comandos de saídas de dados que imprimirão na tela o nome do titular e o saldo atual da conta, respectivamente.

Utilizando Objetos

Note que no código da classe **Programa** foi possível acessar o atributo *nome_titular* e o método *depositar()* da classe **Conta**. Esse acesso foi possível, pois na definição dessa classe **Conta** não definimos um qualificador de acesso para o atributo e nem para o método.

Utilizando Objetos

Logo, tanto o método quanto o atributo são, por padrão, protegidos (*protected*). Além disso, a classe **Conta** está no mesmo pacote que a classe **Programa**. Em breve estudaremos o conceito de encapsulamento e veremos que essa não é uma boa estratégia para controlar os acessos a atributos de uma classe.

Utilizando Objetos

O método *depositar* não retorna nenhum valor para a classe que o chamou. Já o método *sacar* definido na classe **Conta**, por exemplo, retorna um *boolean* que indica se o saque pôde ser efetuado com sucesso ou não.

Utilizando Objetos

Mas, então, como utilizar o valor retornado por um método? A seguir veremos um exemplo em que o método *sacar()* é chamado pelo programa principal e seu retorno é utilizado para exibir uma mensagem informando se o saque foi realizado com sucesso ou não.

Utilizando Objetos

```
1 package exemplos;
2
3 public class Programa {
4
5     public static void main(String[] args) {
6
7         Conta c = new Conta();
8         c.depositar(200);
9         boolean saque_efetuado = c.sacar(250);
10        if (saque_efetuado)
11            System.out.println("Saque Efetuado com Sucesso");
12        else
13            System.out.println("Saque não efetuado! Saldo insuficiente!");
14    }
15 }
```

Utilizando Objetos

Observe que na linha 9 do exemplo anterior é criada uma variável booleana com o nome *saque_efetuado*, que receberá o valor retornado pela chamada ao método *sacar()*. Depois, o valor dessa variável é utilizado para decidir entre exibir a mensagem de “Saque Efetuado com Sucesso” ou a de “Saque não efetuado”.

4 - Atributos e Métodos Estáticos

Atributos e Métodos Estáticos

Atributos estáticos são atributos que contêm informações inerentes a uma classe e não a um objeto em específico. Por isso são também conhecidos como atributos ou variáveis de classe.

Atributos e Métodos Estáticos

Vamos entender uma **variável de classe**. **Variável de Classe** define um atributo de uma classe inteira. A variável se aplica à própria classe e a todas as suas instâncias, de modo que somente um valor é armazenado, não importando quantos objetos dessa classe tenham sido criados.

Atributos e Métodos Estáticos

Para que possamos entender melhor o conceito de atributo estático, vamos voltar ao nosso exemplo do sistema para conta bancária.

Atributos e Métodos Estáticos

Suponha que quiséssemos ter um atributo que indicasse a quantidade de contas criadas. Esse atributo não seria inerente a uma conta em específico, mas a todas as contas. Assim, seria definido como um atributo estático. Para definir um atributo estático em Java, basta colocar a palavra *static* entre o qualificador e o tipo do atributo.

Atributos e Métodos Estáticos

O mesmo conceito é válido para métodos. Métodos estáticos são inerentes à classe e, por isso, não nos obrigam a instanciar um objeto para que possamos utilizá-los.

Atributos e Métodos Estáticos

Para definir um método como estático, basta utilizar a palavra *static*, a exemplo do que acontece com atributos. Para utilizar um método estático devo utilizar o nome da classe acompanhado pelo nome do método.

Atributos e Métodos Estáticos

Métodos estáticos são muito utilizados em classes do Java que proveem determinados serviços. Por exemplo, Java fornece na classe `Math` uma série de métodos estáticos que fazem operações matemáticas como: raiz quadrada (*`sqrt`*), valor absoluto (*`abs`*), seno (*`sin`*) entre outros.

Atributos e Métodos Estáticos

A seguir, podemos observar a aplicação de métodos estáticos em código Java:

Atributos e Métodos Estáticos

```
double x = Math.sqrt(634); //Atribui a x o valor da raiz quadrada de 634  
double z = Math.sin(4); //Atribui a x o valor do seno de 4 radianos  
double y = Math.PI; //Atribui a y o valor da constante matemática pi.
```

Atributos e Métodos Estáticos

No exemplo mostrado anteriormente, podemos ver que não instanciamos objetos da classe *Math* para utilizar seus métodos e constantes, pois são estáticos.

Atributos e Métodos Estáticos

Um método criado como estático só poderá acessar atributos que também sejam estáticos, além dos seus argumentos e variáveis locais.

5 - A Classe *String*

A Classe *String*

Nós sabemos, por estudos já feitos nesta Unidade Curricular, que se quisermos armazenar caracteres especiais em uma variável, devemos declará-la como *char*. Tipo primitivo existente no Java.

A Classe *String*

Mas, e se ao invés de usar apenas um caracter, quiséssemos armazenar uma palavra inteira? Ou seja, um conjunto de caracteres? Qual deveria ser o tipo da variável que armazenaria esta palavra?

A Classe *String*

Para isso, utilizamos em Java a classe ***String***. Esta classe nos permite criar variáveis (objetos instanciados a partir desta classe) que sejam capazes de armazenar uma cadeia de caracteres, ou seja, palavras.

A Classe *String*

É possível também concatenar os objetos do tipo ***String***. Para isto, basta utilizarmos o símbolo “+”.

A Classe *String*

Quando se pretende comparar valores do tipo ***String*** para saber se são iguais utiliza-se o método *equals* e não o símbolo “==”.

A Classe *String*

Existem diversos outros métodos da classe ***String*** muito úteis , a saber:

A Classe *String*

- *length;*
- *charAt;*
- *toUpperCase;*
- *toLowerCase;*
- *trim;*
- *replace;*
- *valueOf.*

A Classe *String*

Vejam os a seguir um exemplo de código com aplicação dos métodos mostrados anteriormente. Rode esses códigos e veja o que acontece no resultado mostrado em tela.

A Classe *String*

```
package exemplos;

public class ExemploString {

    public static void main(String[] args) {
        String nome = "José";
        String frase = " Meu nome é ";
        String completa = frase + nome; //Concateno 2 Strings formando uma nova
        System.out.println(completa + "! ");
        System.out.println("O tamanho do nome é: " + nome.length());
        System.out.println("O caracter da posicao 2 do nome é " + nome.charAt(2));
        System.out.println("Frase Completa toda em maiusculas: " + completa.toUpperCase());
        System.out.println("Substring de 2 a 8: " + completa.subSequence(2, 8));
        System.out.println("Tirando os espaços antes e depois da frase completa: " + completa.trim());
        System.out.println("Substituindo José por João na frase completa: " + completa.replace("José", "João"));
    }
}
```

6 - Listas

Listas

A partir de agora vamos conhecer uma outra estrutura de dados para além dos vetores e das matrizes que já vimos em aulas passadas. Vamos falar das **Listas**.

Listas

A estrutura de dados lista é utilizada para armazenar conjuntos de elementos. A vantagem em utilizar listas em lugar de vetores é o fato de as listas serem alocadas dinamicamente de forma que não precisamos prever seu tamanho máximo.

Listas

Java fornece classes que implementam o conceito de lista. Nesse aula utilizaremos uma dessas classes: o *ArrayList*. Para trabalharmos com a classe *ArrayList* precisamos conhecer seus métodos.

Listas

- *public ArrayList()*: cria um *ArrayList* vazio.
- *public boolean add(<elemento>)*: adiciona um elemento no final da lista.
- *public void add(index, <elemento>)*: adiciona um elemento na posição index.

Listas

- *public <elemento> get(int index)*: obtém o elemento de índice index.
- *public <elemento> remove(int index)*: retorna o elemento de índice index e o elimina da lista.
- *public boolean isEmpty()*: verifica se a lista está vazia.

Listas

Para que possamos navegar ao longo de uma lista, devemos utilizar a **interface** chamada *iterator*. Vejamos os principais métodos desta **interface**:

Listas

- *boolean hasNext()**boolean hasNext()*:
verifica se existe próximo elemento na lista;
- *next()*: obtém o elemento sob o *Iterator* e
avança para o próximo elemento;
- *void remove()*: remove o elemento sob o
Iterator.

Listas

A seguir é apresentado um código que traz a implementação de duas instâncias da classe **Conta** que foram inseridas em uma lista. Neste código também é feita a navegação ao longo da lista para mostrar os números das contas.

Listas

```
package exemplos;

import java.util.ArrayList;
import java.util.Iterator;

public class ExemploLista {

    public static void main(String[] args) {
        ArrayList lista = new ArrayList();
        Conta c = new Conta();
        c.numero = 1;
        lista.add(c);
        c = new Conta();
        c.numero = 2;
        lista.add(0,c);
        Iterator i = lista.iterator();
        while(i.hasNext()){
            c=(Conta)i.next();
            System.out.println("Conta Numero: "+ c.numero);
        }
    }
}
```

Listas

O *ArrayList* pode armazenar objetos de quaisquer tipo. Assim, quando obtemos um objeto ele retorna uma instância do tipo *Object*. Por isso, no código mostrado anteriormente foi necessário fazer a conversão explícita para o tipo **Conta**.

Obrigado!